

Log In ▾

This page is under active construction. Expect bugs.

Modely a architektury paralelních a distribuovaných systémů; prostředky pro jejich implementaci a základní algoritmy.

vector2 ▾

B4B36PDV [Webové stránky předmětu](#) [Nové webové stránky předmětu](#)

• Paralelní systémy/výpočty:

1. **Hardwarová podpora pro paralelní výpočty** – (super)skalární architektury, pipelining, spekulativní vyhodnocování, vektorové instrukce, vlákna, procesy, GPGPU. Hierarchie cache pamětí.
2. **Komplikace v paralelním programování** – souběh (race condition), uváznutí (deadlock), iluze sdílení (false sharing).
3. **Podpora paralelního programování v C a CPP** – pthreads, thread, jthread, atomic, mutex, lock_guard.
4. **Podpora paralelního programování v OpenMP** – sériově-paralelní model uspořádání vláken (fork-join), paralelizovatelná úloha (task region), různé implementace specifikace. Direktivy: parallel, for, section, task, barrier, critical, atomic.
5. **Techniky dekompozice programu** – statické a paralelní rozdělení práce. Threadpool a fronta úkolů. Balancování a závislosti (dependencies).
6. **Techniky dekompozice programu na příkladech:**
 - I. **Řazení** – quick sort, merge sort.
 - II. **Numerická lineární algebra a strojové učení** – násobení matice vektorem, násobení dvou matic, řešení systému lineárních rovnic.

• Distribuované výpočty/systémy:

1. **Úvod do distribuovaných systémů (DS)** – charakteristiky DS, čas a typy selhání v DS.
2. **Detekce selhání v DS** – detektory selhání a jejich vlastnosti.
3. **Čas a kauzalita v DS** – uspořádání událostí v DS, fyzické hodiny a jejich synchronizace, logické hodiny a jejich synchronizace.
4. **Globální stav v DS a jeho výpočet** – řez distribuovaného výpočtu, algoritmus pro distribuovaný globální snapshot, stabilní vlastnosti DS.
5. **Vzájemné vyloučení procesů v DS** – algoritmy pro vyloučení procesů a jejich vlastnosti.
6. **Volba lídra v DS** – algoritmy pro volbu lídra a jejich vlastnosti.

Paralelní systémy/výpočty

1. Hardwarová podpora pro paralelní výpočty

V téhle kapitole se zaměříme na to, jak je moderní hardware navrženy tak, aby zvládal výpočty paralelně – tedy několik operací současně. Ukážeme si různé úrovně a typy paralelismu, od jednoduchého rozdělení instrukcí na více částí (pipelining), přes využití více výpočetních jader (vlákna, procesy), až po extrémní paralelismus na GPU. Pochopení těchto principů je zásadní pro efektivní vývoj výkonných softwarových systémů.

Vektorové instrukce

Často nazývané **SIMD** instrukce – *single instruction, multiple data*.

Vektorové instrukce vykonávají jednu danou operaci na velkém množství dat najednou. Pracují nad speciálními vektorovými registry – až 512bit.

- To znamená, že například můžete násobit dvě celé pole čísel "najednou", místo po jednotlivých prvcích.

- Umožňují značné zrychlení ve výpočetně náročných paralelizovatelných aplikacích, jako např. násobení matic.

Moderní kompilátory jsou schopné automaticky vektorizovat části kódu. Pro znatelné zrychlení je ale často nutná manuální implementace.

Na rozdíl od paralelizace mezi jádry se nemusí řešit problémy synchronizace a latence mezi vlákny. Nevýhoda je často velmi komplikovaná implementace.

Existuje mnoho standardů, nejznámější jsou x86 standardy:

- AVX – starší, velmi rozšířený, až 128bit registry
- AVX2 – novější, přidává nové operace a 256bit registry
- AVX512 – mnoho dalších operací a 512bit registry, méně rozšířený, většinou jen u high-end CPU

Instruction-Level Parallelism (ILP)

- **Definice:** ILP udává, kolik *nezávislých* instrukcí může procesor potenciálně spustit paralelně v rámci *jednoho* programu/vlákná.
- **Zdroj ILP:** nezávislé aritmetické operace, nezávislé paměťové přístupy, nezávislé větvení.
- **Využití ILP:** pipelining, superskalární/VLIW architektury, out-of-order execution.

Pipelining

- **Princip:** Instrukční cesta se dělí na fáze IF → ID → EX → MEM → WB.

To umožňuje, aby různé instrukce byly zpracovávány současně v různých fázích – zvyšuje se **propustnost**, i když latence jednotlivé instrukce zůstává stejná.

- **Teoretický zisk:** až 1 instrukce / takt ($IPC = 1$); limitují ho hazardy a větvení.
- **Superpipelining:** více než 10 stupňů; vyšší frekvence, ale větší penalizace při chybách predikce.
- **Hazardy:**
 - datové – řeší se forwardingem, stally, přejmenováním registrů
 - řídicí – predikce skoků + branch target buffer (BTB)
 - strukturální – řeší se duplikací jednotek

Skalární vs. superskalární procesory

Skalární procesor

- 1 instrukce za takt ($IPC \leq 1$)
- jednodušší architektura
- využívá pipelining

Superskalární procesor

- více instrukcí za takt ($IPC > 1$)
- N-wide architektura (např. 4-wide = až 4 instrukce/takt)
- využívá ILP pomocí paralelního vykonávání více instrukcí

Podtypy superskalární architektury

- **Statická:** paralelně jen instrukce jdoucí v kódu za sebou; při závislostech nastává stall.
- **Dynamická:** out-of-order execution a přejmenování registrů → lepší využití hardwaru.

Shrnutí rozdílů

- **Skalární CPU** ⇒ 1 instrukce/takt, jednoduché, predikovatelné chování.
- **Superskalární CPU** ⇒ více instrukcí/takt, složitější řízení, vyšší výkon.

GPGPU

- **General-Purpose computing on Graphics Processing Units** (GPGPU) využívá původně grafické karty pro obecné výpočty.
- GPU obsahují tisíce jednoduchých jader sdružených do **streaming multiprocessorů (SM)**.

- Ty spouští stovky až tisíce vláken najednou ve skupinách (warp – 32 vláken běžících lock-step).

Architektura a omezení

- **Výpočetní hierarchie:** GPC → SM → block → warp → thread
- **Sdílené prostředky:**
 - shared memory + L1 cache (~128 KB)
 - max 64 warpů / SM (až 2048 vláken)
 - ~255 registrů na vlákno – méně registrů = vyšší obsazenost
- **Paměťová hierarchie:**
 - L1/shared: ~33 cyklů
 - L2: až 2 TB/s (~200 cyklů)
 - HBM2: 1.5 TB/s (~290 cyklů)
 - Host ↔ GPU: PCIe/NVLink – důležitá je minimalizace přenosů

Výpočetní výkon v praxi

* Threadripper 3990X + RTX 4090:

- single-thread: 49 GFLOPS
- multi-thread CPU: 3.7 TFLOPS
- CPU + GPGPU: **100 TFLOPS**

Programovací modely

- **CUDA C/CPP** – pro NVIDIA
- **OpenCL** – otevřený standard
- **SYCL** – moderní CPP model
- **OpenMP target** – direktivy pro GPU

Typické úlohy

- lineární algebra, strojové učení, ray-tracing, simulace, video encoding...

Výhody

- vysoký výkon (tisíce FP jednotek)
- efektivita (TFLOPS/W)
- specializované instrukce (Tensor Cores...)

Nevýhody

- paměťová latence
- omezené registry/shared memory
- přenosy mezi host ↔ device
- warp divergence (větvení)
- vendor lock-in (CUDA)

Shrnutí

GPGPU nabízí **řádově vyšší výkon** než běžné CPU, ale vývoj je složitější. Správné řízení paměti a vláken, spolu s vhodným programovacím modelem, je klíčem k úspěchu.

Vlákna a procesy

- **Proces** je samostatná jednotka běhu programu, má vlastní adresní prostor (paměť), popisovače souborů atd.
 - Spuštění nového procesu je relativně nákladné.
 - Vhodné pro více zcela oddělených výpočetních úloh (např. mikroservisy, více programů najednou).
- **Vlákno (thread)** je lehčí jednotka běhu uvnitř procesu.
 - Sdílí paměť a další prostředky s ostatními vlákny procesu.
 - Přepínání mezi vlákny je levnější než mezi procesy.
 - Vhodné pro paralelizaci uvnitř jedné aplikace – více úloh najednou (např. UI, I/O, výpočty).
 - Pozor na **synchronizaci a souběhy (race conditions)** – nutnost použití zámků, mutexů apod.

Hierarchie cache paměti

Moderní CPU mají vícestupňovou **hierarchii cache**, která zajišťuje rychlý přístup k často používaným datům:

- **L1 cache** – nejmenší (např. 32 KB), ale nejrychlejší (~4 cykly), zvláště pro instrukce a data.
- **L2 cache** – větší (např. 256 KB), pomalejší (~12 cyklů), většinou sdílena mezi jádry.
- **L3 cache** – největší (např. 16 MB), ještě pomalejší (~30–40 cyklů), sdílená mezi všemi jádry.
- **RAM** – mnohem větší, ale o řád(y) pomalejší než cache (~100 ns).
- **Zásada locality**: data, která byla použita nedávno nebo jsou blízko sobě, mají vyšší šanci být v cache – tím se výrazně snižuje průměrná latence paměťových přístupů.

Efektivní využití cache (např. přístup po řádcích, nikoliv po sloupcích v 2D poli) může mít zásadní vliv na výkon.

2. Komplikace v paralelním programování

Když programujeme paralelně – tedy více věcí běží najednou – můžeme narazit na různé složitosti, které se u sekvenčních (jednovláknových) programů vůbec neřeší. Typickými komplikacemi jsou souběh (když dvě vlákna zasahují do stejných dat zároveň), uváznutí (když se navzájem blokují), nebo falešné sdílení (když si vlákna "lezou do cache", i když pracují na různých datech). V téhle kapitole se na tyto problémy podíváme a vysvětlíme si, jak a proč k nim dochází.

Race Condition (souběh)

- **Souběh (race condition)** nastává, když dvě nebo více vláken přistupuje ke **stejným datům** a **alespoň jedno z nich je mění**, přičemž jejich pořadí není řízené (např. zámky, mutexy).
- To znamená, že výsledek programu **závisí na načasování** – někdy to „vyjde dobře“, jindy ne.
- Typickým příkladem je inkrementace sdílené proměnné – pokud není chráněná zámkem, může být výsledek menší, než kolik se očekává.

Například:

```
// špatně – race condition
for (int i = 0; i < 1000; i++) {
    counter++;
}
```

- Řešení:
 - Použít **synchronizační primitiva** – mutexy, atomické operace, semaforey apod.
 - Nebo použít **thread-local** kopie a sloučit je až nakonec.

Deadlock (uváznutí)

- **Uváznutí (deadlock)** nastává, když dvě nebo více vláken čeká na zdroj, který drží to druhé – a žádné se nepohne dál.
- Jednoduše řečeno: vlákno A čeká na zámek, který drží vlákno B, a vlákno B čeká na zámek, který drží vlákno A.

Například:

```
// pseudokód s potenciálním deadlockem
thread1:
lock(A)
lock(B)

thread2:
lock(B)
lock(A)
```

- Aby deadlock mohl nastat, musí být splněny tyto 4 podmínky (tzv. Coffmanovy podmínky):
 - **Vzájemné vyloučení** – zdroje nejsou sdílené (jen jedno vlákno je může mít).
 - **Zadržení a čekání** – vlákno drží jeden zámek a čeká na další.
 - **Neodnímatelnost** – zdroj (zámek) nemůže být násilně odebrán.
 - **Cyklické čekání** – existuje cyklus závislostí mezi vlákny.
- Prevence:
 - Dodržovat **pevné pořadí zamykání zdrojů**.

- Používat **timeouty** u zámeků.
- Využít algoritmy pro **deadlock detection** a recovery.

False Sharing

Nastává, když dvě vlákna přistupují k **různým** proměnným, které se ale nachází ve **stejně cache line**.

- Když jedno vlákno zapíše do své proměnné, CPU invaliduje celou cache line – i když druhé vlákno pracuje na jiné proměnné ve stejné linii.
- Pokud se to děje často, dochází k **velkému zpomalení**, protože cache se neustále synchronizuje mezi jádry.
- Řešení:
 - **Zarovnat proměnné** tak, aby každá byla na vlastní cache line (např. pomocí `alignas(64)`).
 - Přidat „padding“ mezi proměnné, které používají různá vlákna.

True Sharing (není komplikace)

Nastává, když dvě vlákna přistupují ke **stejně** proměnné.

- Narozdíl od false sharing je to **žádoucí chování**, pokud víme, co děláme – ale musí být správně synchronizované.
- Typicky jde o sdílené čítače, fronty, stavové proměnné atd.

3. Podpora paralelního programování v C a CPP

Existuje více úrovní podpory – od nízkoúrovňových knihoven jako `pthread` v C, až po elegantní moderní rozhraní ve stylu `std::thread`, `std::jthread`, `std::mutex` nebo `std::atomic` v CPP. Pokud chceme psát vícevláknové programy efektivně a bezpečně, je důležité pochopit, kdy a jak jednotlivé nástroje použít – a čím se liší.

POSIX vlákna (pthread)

- `pthread` (POSIX Threads) je **standardní rozhraní v jazyku C** pro práci s vlákny.
- Poskytuje funkce pro:
 - **vytváření vláken**: `pthread_create()`
 - **synchronizaci vláken**:
 - **mutexy**: `pthread_mutex_lock()`, `pthread_mutex_unlock()`
 - **podmínkové proměnné**: `pthread_cond_wait()`, `pthread_cond_signal()`
 - **semafory**: `sem_init()`, `sem_wait()`, `sem_post()`

Je to výkonný, ale nízkoúrovňový nástroj – programátor musí vše spravovat ručně. Hodí se pro systémy, kde je důležitá kontrola nad výkonem a kompatibilita se standardem POSIX.

std::thread (C++11)

- Od C++11 je k dispozici standardní knihovna pro vlákna – `std::thread`.
- Umožňuje jednoduché vytvoření a správu vláken pomocí objektově orientovaného rozhraní.
 - Vlákno spustíme předáním funkce (nebo lambdy) do konstruktoru.
 - Synchronizace se řeší pomocí `std::mutex`, `std::condition_variable` a dalších primitiv.

Výhodou oproti `pthread` je **vyšší přehlednost, typová bezpečnost a integrace s C++ idiomy** (např. RAII, STL, lambda).

std::jthread (C++20)

- Od C++20 přibyla nová třída `std::jthread`, která automaticky spravuje vlákno.
 - Při zničení objektu se vlákno **automaticky ukončí** (`join()` nebo `detach()`).
 - Má vestavěnou podporu pro **zrušení vlákna** pomocí `stop_token`.
 - Výborně se hodí pro práci s **RAII přístupem** a bezpečnější práci s vláknem.

To znamená, že se snižuje riziko zapomenutého `join()` a tím i potenciálních chyb.

Mutex a Lock Guard

Mutex (*mutual exclusion*) je synchronizační nástroj, který **zajišťuje, že jen jedno vlákno přistupuje k určité části kódu najednou** – typicky tzv. *kritické sekci*.

- V CPP se používá `std::mutex` a jeho varianty:
 - `std::recursive_mutex`, `std::timed_mutex`, ...

Ukázka (ruční zamykání/odemykání):

```
#include <iostream>
#include <thread>
#include <mutex>

int main() {
    std::mutex mtx;

    auto threadFunc = [&mtx]() {
        mtx.lock();
        // Kritická sekce
        std::cout << "Thread is executing critical section." << std::endl;
        mtx.unlock();
    };

    std::thread t1(threadFunc);
    std::thread t2(threadFunc);

    t1.join();
    t2.join();

    return 0;
}
```

Moderní CPP doporučuje použít `std::lock_guard`, který mutex **automaticky zamkne a odemkne** – tím se předejde chybám (např. zapomenutí `unlock()` při výjimce):

```
#include <iostream>
#include <thread>
#include <mutex>

int main() {
    std::mutex mtx;

    auto threadFunc = [&mtx]() {
        std::lock_guard<std::mutex> lock(mtx);
        // Kritická sekce
        std::cout << "Thread is executing critical section." << std::endl;
    };

    std::thread t1(threadFunc);
    std::thread t2(threadFunc);

    t1.join();
    t2.join();

    return 0;
}
```

Atomic

Atomic proměnné umožňují provádět operace, které jsou **nedělitelné (atomické)** – tím pádem **bezpečné vůči souběhu**, aniž by bylo potřeba používat mutex.

- V CPP se používá `std::atomic<T>` – kde **T** může být např. `int`, `bool`, pointer apod.
 - V pozadí to využívá **atomické instrukce CPU** (pokud jsou dostupné), takže jsou **rychlejší než mutexy**.
 - Pokud HW atomické instrukce neumí, fallback je přes mutex.

V praxi jsou tedy `std::atomic` velmi efektivní pro jednoduché sdílené proměnné – například čítače:

```
#include <iostream>
#include <thread>
#include <atomic>

int main() {
    std::atomic<int> counter(0);

    auto increment = [&counter]() {
        for (int i = 0; i < 1000; ++i) {
```



```

        ++counter;
    }
};

std::thread t1(increment);
std::thread t2(increment);

t1.join();
t2.join();

std::cout << "Final counter value: " << counter.load() << std::endl;

return 0;
}

```

- Jinými slovy – `std::atomic` je ideální, pokud chceš **rychlou synchronizaci bez složitých zámků**, ale nepotřebuješ složitou logiku jako čekání, notifikace, podmínky apod.

4. Podpora paralelního programování v OpenMP

OpenMP je velmi populární knihovna pro paralelizaci v jazycích C, CPP a Fortran. Umožňuje snadno spouštět více vláken pomocí direktiv vkládaných přímo do zdrojového kódu. V této kapitole se podíváme na to, jak funguje základní model **fork-join**, jak se v OpenMP definují úlohy (tasks), a jaké máme k dispozici prostředky pro synchronizaci. Ukážeme si i různé varianty implementací OpenMP a přehled direktiv, které programátorům umožňují jemné řízení paralelismu.

Sériově-paralelní model uspořádání vláken (fork-join)

Fork-join model je základní exekuční schéma OpenMP. Program začíná jedním **master** vláknem, které po vstupu do direktivy **parallel** vytvoří tým dalších vláken (**fork**). Všechna vlákna spolupracují v daném paralelním regionu a na jeho konci se implicitní bariérou zase spojí do jediného vlákna (**join**).

- Výhody:
 - jednoduchá správa vláken
 - sdílená paměť
 - předvídatelné chování a synchronizace

Ukázkový kód:

```

#include <omp.h>
#include <stdio.h>

int main(void) {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int n = omp_get_num_threads();
        printf("Hello from thread %d of %d\n", id, n);
    } /* <-- implicit join & barrier */
    return 0;
}

```

- Poznámka: OpenMP vytváří **vlákna**, nikoliv procesy – všechna sdílí paměť. Vlákna jsou obvykle implementována pomocí POSIX/threads.

Další možnosti a doplňky modelu:

- **#pragma omp barrier** – ruční synchronizační bod.
- **single, master** – sekci vykoná pouze jedno vlákno.
- **ordered** – zachování pořadí iterací ve smyčce.
- **critical, atomic** – zajištění sériového přístupu ke sdíleným proměnným.
- **nowait** – zabráni implicitní bariéře na konci paralelního bloku.
- **OMP_NESTED=TRUE** nebo **omp_set_max_active_levels()** – aktivuje vnořenou paralelizaci.
- **task, taskgroup, depend(...)** – jemné řízení závislostí mezi úlohami (viz níže).

Paralelizovatelná úloha (task region)

Pomocí direktivy `task` lze definovat úlohy, které mohou být nezávisle spouštěny paralelně. Oproti `parallel for`, které rozděluje smyčku, `task` umožňuje paralelizovat **libovolné části kódu**, včetně rekurze a nepravidelné struktury výpočtu.

- Tasky lze organizovat do `taskgroup`.
- Pomocí `depend(in|out|inout: var)` lze řídit pořadí a závislosti.
- `detach`, `priority` – umožňují pokročilou kontrolu provádění.

Příklad:

```
#pragma omp task depend(out: A) {  
    // vypočítáme A  
}  
  
#pragma omp task depend(in: A) {  
    // použijeme A  
}
```

Různé implementace OpenMP specifikace

OpenMP je pouze specifikace – jednotlivé kompilátory ji implementují v různé míře.

Přehled nejznámějších implementací (stav 2025):*

- **GCC 14+ (libgomp)** – `-fopenmp`, verze 5.2 (téměř úplná), open-source klasika.
- **Clang 18+ (libomp)** – `-fopenmp`, verze 5.2 (většina), funguje i na Windows.
- **Intel oneAPI (icx/icpx + iomp5)** – `-qopenmp`, verze 5.2, velmi výkonný runtime, podpora GPU.
- **IBM XL (LLVM-based)** – `-qsmp=omp`, verze 5.1, optimalizace pro POWER.
- **AMD AOCC 4.x** – laděno pro AMD Zen, Clang + libomp.
- **Cray/ROCm** – zaměřeno na GPU/offload (OpenMP target).

Přehled hlavních direktiv OpenMP

`#pragma omp parallel`

- Vytvoří tým vláken.
- Podporuje: `num_threads`, `if`, `default`, `shared`, `private`, ...

`for / do`

- Paralelizace smyčky.
- Možnosti: `schedule(static|dynamic|guided)`, `collapse(n)`, `reduction`, `nowait`.

`sections / section`

- Spustí nezávislé bloky kódu paralelně (např. různé funkce najednou).

`task`

- Vytvoří úlohu; možnost specifikace závislostí přes `depend(...)`.
- Další klauzule: `priority`, `detach`.

`barrier`

- Explicitní synchronizační bod – všechna vlákna musí dojít na toto místo.

`critical [(name)]`

- Zamezí více vláknům vstup do stejné sekce kódu najednou.
- Jméno odděluje různé nezávislé sekce.

`atomic`

- Odlehčená synchronizace pro jednoduché operace s proměnnou (inkrementace, zápis, čtení...).

Příklad zapojení všeho dohromady

```
#include <omp.h>
#include <stdio.h>

int main(void) {
    #pragma omp parallel
    #pragma omp single // jeden thread spouští tasky
    {
        #pragma omp task depend(out: sum)
        {
            int sum = 0;

            #pragma omp parallel for reduction(+ : sum) nowait
            for (int i = 0; i < 1e6; ++i)
                sum += i;

            printf("Sum done: %d\n", sum);
        }

        #pragma omp task depend(in: sum)
        {
            printf("Post-processing...\n");
        }
    } // implicit taskgroup – čeká se na všechny tasky
    return 0;
}
```

Tento příklad ukazuje vytvoření tasku se závislostí `depend(out: sum)`, paralelní redukci vnitřní smyčky, a následný task, který čeká na výsledek (`depend(in: sum)`).

5. Techniky dekompozice programu

Paralelní programy často potřebují být rozděleny na menší části – a to nejen podle funkcí, ale hlavně podle **práce**, kterou je třeba vykonat. Tato kapitola se věnuje technikám, jak takovou práci rozdělit mezi vlákna nebo úlohy (tasky), a jak přitom řešit problémy jako nevyvážená zátěž nebo závislosti mezi úlohami. Pochopení těchto principů ti pomůže psát efektivnější a škálovatelnější paralelní kód.

Statické x dynamické rozdělení práce

- **Statické rozdělení** – práce (např. iterace smyčky) se rozdělí předem mezi všechna vlákna.
 - Např. pomocí `schedule(static[, chunk])` v OpenMP.
 - Výhodou je **nízký overhead** (prakticky žádná synchronizace).
 - Nevýhodou je, že **u nepravidelných úloh může být některé vlákno přetížené**, zatímco jiné skončí dříve a zahálí.
- **Dynamické rozdělení** – práce se přiděluje **za běhu** podle toho, které vlákno je volné.
 - Používá se `schedule(dynamic[, chunk])`.
 - Výhodou je lepší **vyvážení zátěže**.
 - Nevýhodou je **vyšší režie kvůli synchronizaci**.
- **Guided plánování** – kompromis mezi statickým a dynamickým:
 - `schedule(guided[, chunk])` začíná velkými bloky a postupně zmenšuje chunk velikost.
 - Vhodné pro případy, kdy práce trvá různě dlouho a časem ubývá.
- Další možnosti:
 - `schedule(runtime)` – výběr plánování je ponechán na hodnotě proměnné `OMP_SCHEDULE`.
 - `schedule(auto)` – nechá výběr plánu na implementaci OpenMP.

Thread-pool a fronta úkolů (work-stealing)

- OpenMP (od verze 3.0) podporuje **tasky**, které jsou plánovány na vlákna z **persistentního thread-poolu**.
 - Vlákna jsou vytvořena v prvním paralelním regionu a **zůstávají aktivní** i pro další úseky programu.
- Každé vlákno má vlastní **lokální deque** (obousměrnou frontu), do které si vkládá nové úkoly.
- **Work-stealing**: pokud vlákno nemá co dělat, zkusí si „ukrást“ úkol z cizí fronty.
 - Tento přístup pomáhá **automaticky vyvažovat zátěž**.
 - Push/pop na vlastní frontě jsou většinou **lock-free**; zámky se používají jen při krádeži.

Balancování zátěže

Vyrovnané rozdělení práce je klíčem k výkonu paralelního programu.

- U **smyček**:
 - Volba správného `schedule(...)` plánu.
 - Velikost `chunku` ovlivňuje granularitu přidělené práce.
 - `collapse(n)` – slučuje více vnořených smyček do jedné, čímž zvyšuje počet iterací pro paralelizaci.
- U **tasků**:
 - Work-stealing pomáhá s dynamickým plánováním.
 - Pomocí `priority(expr)` lze zvýhodnit důležité úkoly.
 - Heuristiky runtime mohou **omezit vznik malých tasků** (např. jejich inline-ing) → lepší škálování.

Závislosti (dependencies)

Tasky mohou mít mezi sebou **závislosti**, které určují pořadí provádění. V OpenMP se deklarují pomocí `depend(...)`.

- Typy závislostí:
 - `depend(in: X)` – task **potřebuje** data `X`.
 - `depend(out: X)` – task **produkuje** data `X`.
 - `depend(inout: X)` – task `X` čte i zapisuje.
 - Další pokročilé typy: `mutexinoutset`, `depobj` – pro specializované scénáře.

Ukázkový kód:

```
#pragma omp task depend(out:A)
build_A();

#pragma omp task depend(in:A) depend(out:B)
build_B();

#pragma omp task depend(in:B)
use_B();
```

- OpenMP runtime si z těchto závislostí **sestaví DAG (acyklický graf)**.
- Jakmile jsou všechny vstupní závislosti splněny, task je označen jako „**ready**“ a zařadí se do fronty.

Tato metoda umožňuje velmi jemné řízení výpočtu a paralelizace i **nepravidelných nebo stromových struktur** (např. kompilátory, plánovače, grafy).

6. Techniky dekompozice programu na příkladech

Abychom lépe pochopili různé způsoby paralelizace, ukážeme si je na konkrétních příkladech z oblasti řazení, lineární algebry a strojového učení. Uvidíme, jak lze algoritmy jako quick sort nebo násobení matic přirozeně rozdělit do paralelních částí a jaké techniky přitom použít – např. tasky, `collapse`, blokové zpracování či závislosti.

Řazení

quick sort a merge sort

Quick sort

- Proč „rozděl a panuj“?
 - ***Divide***: vybereme pivot a jedním průchodem rozdělíme pole na část $< \text{pivot}$ a část $\geq \text{pivot}$.
 - ***Conquer***: obě části rekurzivně řadíme – ideální místo pro vytvoření dvou paralelních úloh (*tasků*).
 - ***Combine***: není třeba – dělení samo zajistí správné pořadí.
- Paralelizace: po operaci `partition` spustíme levé větvení jako `omp task`, pravé běží ve vlákne dál.

```
void qs(std::vector<int>& vec, int from, int to) {
    if (to - from <= base_size) {
        std::sort(vec.begin() + from, vec.begin() + to);
        return;
    }

    int pivot = vec[from];
```

```

    int mid = partition(vec, from, to, pivot); // divide

    if (mid - from > 1) { // conquer - left
#pragma omp task shared(vec) firstprivate(from, mid)
        qs(vec, from, mid);
    }
    if (to - mid > 1) { // conquer - right
        qs(vec, mid, to);
    }
}

```

Merge sort

• Proč „rozděl a panuj“?

- ***Divide***: rekurzivní dělení pole na půlky.
- ***Conquer***: každou půlku řadíme paralelně.
- ***Combine***: dvě seřazené části spojíme lineárním **merge**.
- Paralelizace: obě rekurze běží jako tasky, po jejich dokončení proběhne **inplace_merge**.

```

void ms(std::vector<int>& vec, int from, int to) {
    if (to - from <= base_size) {
        std::sort(vec.begin() + from, vec.begin() + to); // seriál-insert
        return;
    }
    int mid = from + (to - from) / 2; // divide

#pragma omp task shared(vec) firstprivate(from, mid)
    ms(vec, from, mid); // conquer - left
    ms(vec, mid, to); // conquer - right

#pragma omp taskwait
    std::inplace_merge(vec.begin() + from, // combine
                      vec.begin() + mid,
                      vec.begin() + to);
}

```

Numerická lineární algebra a strojové učení

násobení matice × vektor, násobení dvou matic, řešení systému lineárních rovnic

Paralelní násobení matice × vektor (SpMV/Dense MV)

- **Princip**: každý prvek y_i je skalární součin řádku A_i a vektoru x .
 - ***Divide***: rozdělíme po řádcích.
 - ***Conquer***: každý řádek počítá vlákno nezávisle.
 - ***Combine***: nepotřebujeme – každý výstupní $y[i]$ je unikátní.
- U husté matice: nejlepší přístup po řádcích (row-major).
- U řídké: paralelizace přes nenulové řádky (např. CSR formát).

```

void matvec_dense(const std::vector<double>& A, // ROWS × COLS, row-major
                 const std::vector<double>& x,
                 std::vector<double>& y) {
#pragma omp parallel for
    for (int i = 0; i < ROWS; ++i) {
        double sum = 0.0;
        for (int j = 0; j < COLS; ++j)
            sum += A[i * COLS + j] * x[j]; // dot product
        y[i] = sum;
    }
}

```

Paralelní násobení dvou matic

- **Naivní přístup**: každý prvek c_{ij} samostatně – špatná cache-lokalita.
- **Blokové (tiling) násobení**:
 - ***Divide***: rozdělíme výstupní matici C na bloky B_{pq} .
 - ***Conquer***: každý blok počítá vlákno/task – např. pomocí **collapse(2)**.
 - ***Combine***: není třeba – bloky jsou oddělené.

```

#pragma omp parallel for collapse(2) schedule(dynamic)
for (int bi = 0; bi < N; bi += BS)
    for (int bj = 0; bj < N; bj += BS)
        for (int bk = 0; bk < N; bk += BS)
            multiply_block(A, B, C, bi, bj, bk, BS);

```

- Pro vysoký výkon: používat BLAS knihovny (OpenBLAS, MKL...) se SIMD a tilingem.

Paralelní řešení systému lineárních rovnic (Gaussova eliminace)

- **Gaussova eliminace (LU):** v kroku k odstraňujeme hodnoty pod pivotem A_{kk} .
 - Paralelizujeme řádky $i = k + 1..n - 1$.
 - Nutná synchronizace mezi kroky – `taskwait` nebo sekvenční závislosti.

```
void gauss_elim(std::vector<double>& A) { // ROWS x COLS, row-major
    for (int k = 0; k < ROWS - 1; ++k) {
        #pragma omp parallel for
        for (int i = k + 1; i < ROWS; ++i) {
            double c = -A[i * COLS + k] / A[k * COLS + k];
            for (int j = k; j < COLS; ++j)
                A[i * COLS + j] += c * A[k * COLS + j];
        }
    }
}
```

- **Iterativní metody (Jacobi, CG, GMRES):**
 - Lépe paralelizovatelné než Gauss – hlavně `matvec`, vektorové operace, redukce.
 - OpenMP poskytuje `reduction(+:)` a `atomic` pro synchronizaci.
- **Off-load na GPU (OpenMP 5.x):**
 - Pomocí `target teams distribute parallel for` lze výpočet přenést na GPU.
 - Host kontroluje synchronizaci, ale výpočet běží plně na zařízení.

Distribuované výpočty/systemy

1. Úvod do distribuovaných systémů (DS)

Distribuované systémy (DS) jsou o tom, jak spolupracují **nezávislé výpočetní jednotky**, které spolu **nesdílejí paměť ani hodiny**, ale komunikují pomocí zpráv. Jsou všude kolem nás – od cloudových služeb po blockchain nebo velké paralelní výpočty. Tato kapitola představuje základní charakteristiky DS, problematiku času a přehled typických typů selhání, se kterými musí každý návrh systému počítat.

Distribuovaný systém je:

- soustava **autonomních procesů** spojených sítí,
- komunikace probíhá **pouze zprávami** (message passing),
- **žádná sdílená paměť, žádné globální hodiny**,
- procesy mohou **selhávat nezávisle**,
- každý proces má **lokální hodiny**, které mohou běžet různou rychlostí.

Kvůli těmto vlastnostem je velmi těžké se v systému spolehlivě orientovat v čase – nemůžeme dát pevný limit na doručení zprávy nebo délku výpočtu.

Asynchronní x Synchronní DS

Tyto modely popisují „jaký svět“ předpokládáme při návrhu distribuovaného algoritmu – tedy **jaké vlastnosti sítí, hodin a chování procesů** budeme brát jako dané.

Synchronní model

- Známe **horní hranice** pro:
 - zpoždění zpráv,
 - rychlost výpočtu,
 - odchylku mezi hodinami.
- Můžeme navrhovat algoritmy ve **fázích (kolech)** a spolehlivě používat timeouty.
- Takto předvídatelné sítě se v praxi téměř **nevyskytují** – je to spíš teoretický model.

Asynchronní model

- **Žádné záruky** – zpráva může zůstat „viset“ libovolně dlouho, proces se může libovolně zdržet, hodiny se mohou rozjet.

- Timeouty **nejdou spolehlivé** – nelze říct, jestli se proces „jen zpozdil“ nebo opravdu „spadl“.
- Například slavný důkaz **FLP** ukazuje, že **v asynchronním modelu není možný deterministický konsensus**, pokud může padnout byť jen jeden proces.

Částečně synchronní model

- Realistický kompromis:
 - Systém se může nějakou dobu chovat asynchronně, **ale nakonec se „ustálí“** a začne dodržovat limity.
- Většina reálných protokolů (např. **Raft**, **PBFT**) počítá právě s tímto modelem.
- Výhoda: **odolnost vůči výpadkům i výkyvům**, ale **zaručená dohoda** nakonec proběhne.

Typy selhání v distribuovaných systémech

V DS musíme počítat s tím, že **něco selže** – a často i nepozorovaně. Rozlišujeme:

- **Selhání procesu:**
 - *crash / fail-stop* – proces přestane fungovat (nic už neposílá).
 - *byzantské (libovolné) selhání* – proces dál funguje, ale **chybně** (např. posílá nesmysly, nebo se chová škodlivě).
- **Selhání kanálu:**
 - *ztráta zprávy* – zpráva se ztratí v síti.
 - *partitioning* – síť se rozdělí na **oddělené oblasti**, které spolu **nemohou komunikovat**.
- **Selhání časování:**
 - odezva procesu nebo zprávy překročí **očekávané časové limity** – může jít o zpoždění i výpadek.

Předpoklady na komunikační kanál

Distribuované algoritmy často **předpokládají vlastnosti komunikační vrstvy** – obvykle se uvažuje tzv. „ideální, ale pomalá“ síť:

- **spolehlivé doručení** – žádné ztráty zpráv,
- **žádné duplikace** – zprávy se nedoručují víckrát,
- **žádné falešné zprávy** – proces nemůže dostat zprávu, kterou nikdo neposlal,
- **zachování pořadí** – zprávy mezi dvěma procesy dorazí ve stejném pořadí, v jakém byly odeslány.

Tato vlastnosti se často **simulují vyšší vrstvou (např. TCP)**, ale ne vždy – a je třeba s tím počítat při návrhu protokolu.

2. Detekce selhání v DS

V distribuovaných systémech je běžné, že některé procesy selžou – a aby systém mohl dál správně fungovat, ostatní uzly se to musí nějak **včas a spolehlivě dozvědět**. K tomu slouží tzv. **detektory selhání**, které monitorují ostatní uzly (např. pomocí heartbeat zpráv) a označují ty, které nereagují, jako havarované.

Jenže v distribuovaném prostředí nemáme jistotu, jestli uzel selhal, nebo je jen dočasně pomalý (např. síťová prodleva) – a proto je detekce selhání **zásadně nejistá**. Tato kapitola rozebírá vlastnosti detektorů selhání, různé typy heartbeat protokolů a praktický algoritmus SWIM.

Vlastnosti detektorů selhání

- **Úplnost (completeness)**
 - Každé skutečné selhání je **časem detekováno** alespoň jedním bezchybným uzlem.
- **Přesnost (accuracy)**
 - Detektor **nemá falešné poplchy** – neoznačí proces za havarovaný, pokud ve skutečnosti běží.

Nelze zaručit obě vlastnosti zároveň (dle FLP výsledku) – proto většina systémů **upřednostňuje úplnost**, i když občas dojde k mylné detekci.

Jinými slovy: **lepší je omylem někoho považovat za mrtvého, než si nevšimnout, že skutečně spadl**.

- Frekvence selhání v praxi roste **lineárně s počtem uzlů** – tj. v systému se stovkami uzlů je běžné, že některé z nich selžou v každém okamžiku.

Průběh detekce

1. Proces p_j selže.
2. Jiný proces p_k jeho selhání **zjistí** (např. nepřišel heartbeat).
3. Proces p_k **šíří informaci** o selhání p_j dalším uzlům – podle typu protokolu.

Typy detekčních protokolů

Centralizovaný heartbeat

- Každý uzel **periodicky posílá heartbeat** jednomu vybranému procesu p_j každých $\mathcal{T}$ jednotek času.
- p_j si udržuje čítač a čas posledního přijetí od každého uzlu.
- Pokud nedorazí heartbeat během $\tau \rightarrow p_i$ je považován za selhaný.

Vlastnosti:

- Jednoduchá implementace.
- **Nezjistí selhání p_j samotného** – není nikdo, kdo by ho hlídal.
- p_j může být **přetížen**, pokud systém obsahuje hodně uzlů.

Kruhový heartbeat

- Každý proces periodicky posílá heartbeat **sousedovi** v kruhu.
- Lze použít **jednosměrný** nebo **obousměrný** kruh.

Vlastnosti:

- Není žádný centrální bod $\rightarrow$ lepší škálování.
- **Není úplný**, pokud selže jeden (v jednosměrném) nebo dva (v obousměrném) uzly.
- Nutno **udržovat kruhovou topologii** – složitější údržba.

All-to-all heartbeat

Každý uzel **periodicky odesílá heartbeat všem ostatním**.

Vlastnosti:

- **Vysoká úplnost** – každý je hlídán všemi.
- Rovnoměrná zátěž mezi uzly.
- **Nízká přesnost** – síťové zpoždění může vést k falešnému označení za mrtvého.
- **Velké zatížení sítě** – škáluje špatně pro stovky uzlů.

SWIM protokol

Scalable Weakly-consistent Infection-style Membership

Moderní přístup k detekci, který **škáluje a je adaptivní**.

Každý proces p_i periodicky:

1. Posílá **ping** náhodnému uzlu p_j a čeká na **ack**.
2. Pokud žádná odpověď nepřijde, požádá $\mathcal{K}$ jiných uzlů, aby se zeptaly místo něj (**ping_req**).
3. Tyto uzly pingnou p_j , a pokud odpověď dostanou, pošlou ji zpět p_i **jako** **ping_ack**.
4. Pokud nikdo z $\mathcal{K}$ nedostane odpověď $\rightarrow p_j$ je označen za mrtvého.

Vlastnosti:

- **Přesnost** lze nastavit volbou $\mathcal{K}$ – roste s $\mathcal{K}$, chybovost klesá exponenciálně.
- **Úplnost** je zaručena.
- **Průměrný čas detekce:**
- $\frac{e}{e-1} \mathcal{T}$

- průměrná délka detekčního cyklu v SWIM protokolu
- Dobře funguje i ve velkých systémech (1000+ uzlů).

3. Čas a kauzalita v DS

V běžném programu máme přesné hodiny, které říkají, kdy se co stalo. V distribuovaném systému ale žádné „společné“ hodiny nemáme – každý uzel má své vlastní a běží jinak rychle. Proto je v DS mnohem důležitější **pořadí a závislosti událostí** než konkrétní čas. V této kapitole si vysvětlíme, jak fungují fyzické a logické hodiny, co znamená synchronizace a jak se sleduje kauzalita mezi událostmi.

Problémy s časem v DS

- **Clock slew** – rozdíl ve skutečném čase mezi dvěma procesy.
- **Clock drift** – rozdíl v rychlosti běhu hodin (jeden proces má hodiny „rychlejší“ nebo „pomalejší“ než druhý).
- To znamená, že hodiny **nelze úplně sladit**, jen přiblížit.

Synchronizace hodin

Synchronizace může být:

Externí synchronizace

- Každý proces p_i má své hodiny $\mathcal{C}_i$ udržované v rozmezí δ od nějakého **externího referenčního času** $\mathcal{S}$.
 - Např. atomové hodiny, UTC, NTP servery.
 - $|\mathcal{C}_i - \mathcal{S}| \leq \delta$
 - Každý uzel se snaží držet v intervalu δ od globálního času
- Typický příklad: **NTP (Network Time Protocol)**.

Interní synchronizace

- Zajišťuje, že hodiny **každého páru procesů** se liší nanejvýš o δ .
 - Formálně: $|\mathcal{C}_i - \mathcal{C}_j| \leq \delta$
 - Rozdíl mezi hodinami dvou uzlů je nejvýše δ
- Není vázaná na žádný „reálný čas“, ale zajistí, že uzly se navzájem „drží při sobě“.
- Např. **Berkeley algorithm**.

Praktické algoritmy pro synchronizaci

Cristianův algoritmus

- Klient se zeptá serveru: „Kolik je hodin?“
- Server odpoví, a klient si upraví svůj čas podle:

$$\mathcal{C}_i := t + \frac{\mathcal{T}_{RT} - l_{\min} - l'_{\min}}{2}$$

- RTT mínus odhadnutá minimální latence tam i zpět, děleno dvěma

Očekávaná **chyba synchronizace** je: $\leq \frac{\mathcal{T}_{RT} - l_{\min} - l'_{\min}}{2}$

Lokální čas lze zvyšovat okamžitě, ale **nelze ho vracet zpět** – místo toho se mění rychlost přibývání.

NTP

- Servery tvoří **stromovou hierarchii** (stratum 0, 1, 2, ...).
- Uzly synchronizují čas s rodiči i některými sousedy.
- Latence se odhaduje pomocí offsetu:

$$o = \frac{(t_1^r - t_2^r + t_2^s - t_1^s)}{2}$$

- vypočtený odhad offsetu mezi hodinami klienta a serveru

Logické hodiny a kauzalita

Logické hodiny nejsou o „skutečném čase“, ale o **pořadí událostí**. Nepotřebují žádnou synchronizaci – pouze konzistentní značkování událostí podle „co mohlo ovlivnit co“.

Kauzální vztah – relace „stalo se před“ ($\rightarrow$)

Událost $\mathcal{A}$ mohla ovlivnit událost $\mathcal{B}$, pokud:

- $\mathcal{A} \rightarrow \mathcal{B}$ – pokud obě nastaly ve stejném procesu a $\mathcal{A}$ byla dřív,
- nebo $\mathcal{A}$ je odeslání zprávy a $\mathcal{B}$ je její přijetí,
- nebo (tranzitivně): $\mathcal{A} \rightarrow \mathcal{B} \rightarrow \mathcal{C} \Rightarrow \mathcal{A} \rightarrow \mathcal{C}$

Kauzální nezávislost

- Události e_1, e_2 jsou **současné** (nezávislé), pokud:

$$e_1 \parallel e_2$$

- Jinými slovy: žádná z nich nemohla ovlivnit druhou.

Lamportovy logické hodiny

Každý proces si udržuje **číselnou hodnotu svých hodin**:

- Při každé lokální události: $\mathcal{C}_i := \mathcal{C}_i + 1$
- Při odeslání zprávy m : zpráva dostane časovou značku $ts(m) := \mathcal{C}_i$
- Při přijetí zprávy m procesem p_j :

$$\mathcal{C}_j := \max(\mathcal{C}_j, ts(m)) + 1$$

Důležité:

- Jestliže $e_1 \rightarrow e_2$, pak $\mathcal{C}(e_1) < \mathcal{C}(e_2)$
- Ale: $\mathcal{C}(e_1) < \mathcal{C}(e_2)$ **neznamená**, že $e_1 \rightarrow e_2$

$\Rightarrow$ Lamportovy hodiny **respektují kauzalitu**, ale **neumí ji zpětně ověřit**.

Vektorové hodiny

Abychom **přesně zachytili kauzalitu**, každý proces si udržuje **vektor hodin**, kde každý prvek reprezentuje „jaký čas vím o ostatních“:

- Vektor V_i proces p_i má délku n (počet procesů).
- Při každé lokální události: $V_i[i] + 1$
- Při odeslání zprávy se posílá kopie V_i
- Při přijetí zprávy m se provede prvek po prvku:

$$V_i[j] := \max(V_i[j], V_m[j]) \quad \text{pro všechna } j$$

a poté $V_i[i] + 1$

Událost e_1 předchází e_2 **ve vektorovém čase**, pokud:

$$V(e_1)[k] \leq V(e_2)[k] \quad \forall k \text{ a alespoň jedna nerovnost je ostrá}$$

To už umí odhalit **kauzální závislost i nezávislost**.

4. Globální stav v DS a jeho výpočet

V distribuovaném systému neexistuje globální hodina ani centrální pohled na „okamžitý stav celého systému“. Přesto někdy potřebujeme **získat konzistentní snímek stavu** všech procesů a kanálů – třeba pro detekci uváznutí, garbage collection nebo checkpointing. Tato kapitola vysvětluje, co znamená **řez distribuovaného výpočtu**, jak funguje **Chandy-Lamportův algoritmus** pro získání globálního snapshotu a co jsou **stabilní vlastnosti** systému.

Co je globální stav a snapshot

Globální stav je:

- množina **lokálních stavů všech procesů** +
- stav **všech komunikačních kanálů**
- ... ve stejném (logickém) okamžiku

Globální snapshot je **záznam** takového stavu, který může být analyzován mimo běžící systém (např. logicky, offline).

Použití:

- **garbage collection** – rozhodnutí, že objekt už není dosažitelný
- **detekce deadlocku**
- **ukončení výpočtu**
- **checkpointing** – ukládání stavu pro případ obnovení

Řez distribuovaného výpočtu

Řez (cut) definuje:

- **časový okamžik** v každém procesu a
- **hranici** mezi tím, co se „už stalo“, a tím, co „teprve nastane“

Události, které nastanou **před řezem**, do něj patří; zbytek je mimo.

Konzistentní řez

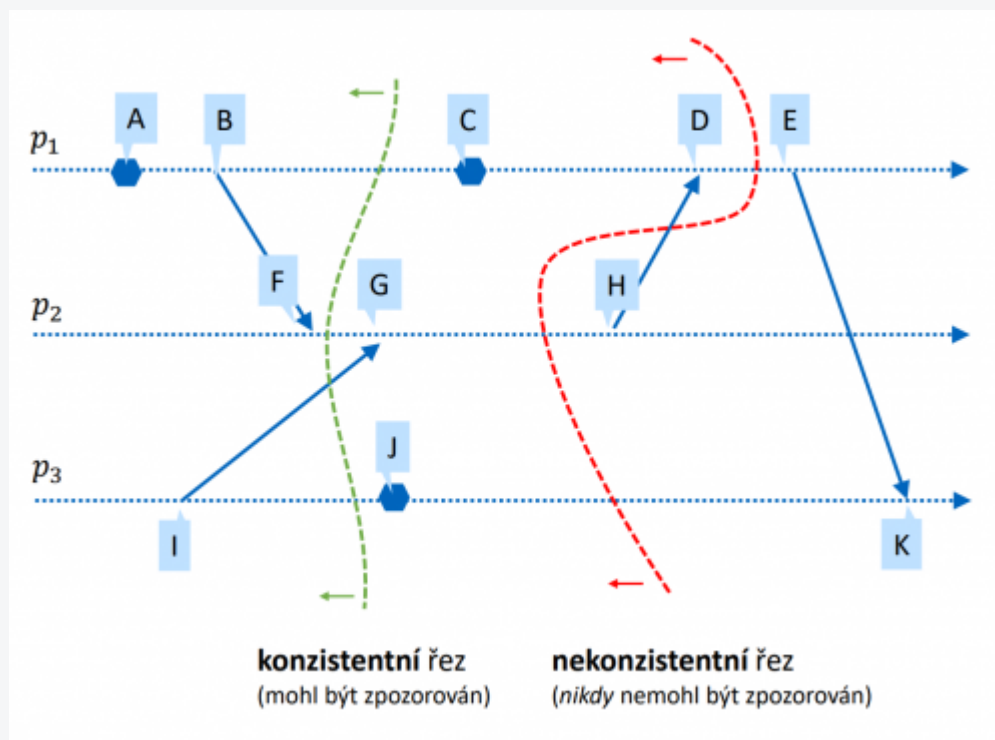
Řez $\tilde{\mathcal{R}}$ je **konzistentní**, pokud:

$$f \in \tilde{\mathcal{R}} \wedge e \rightarrow f \implies e \in \tilde{\mathcal{R}}$$

Jinými slovy: **jestliže obsahuje důsledek, musí obsahovat i příčinu.**

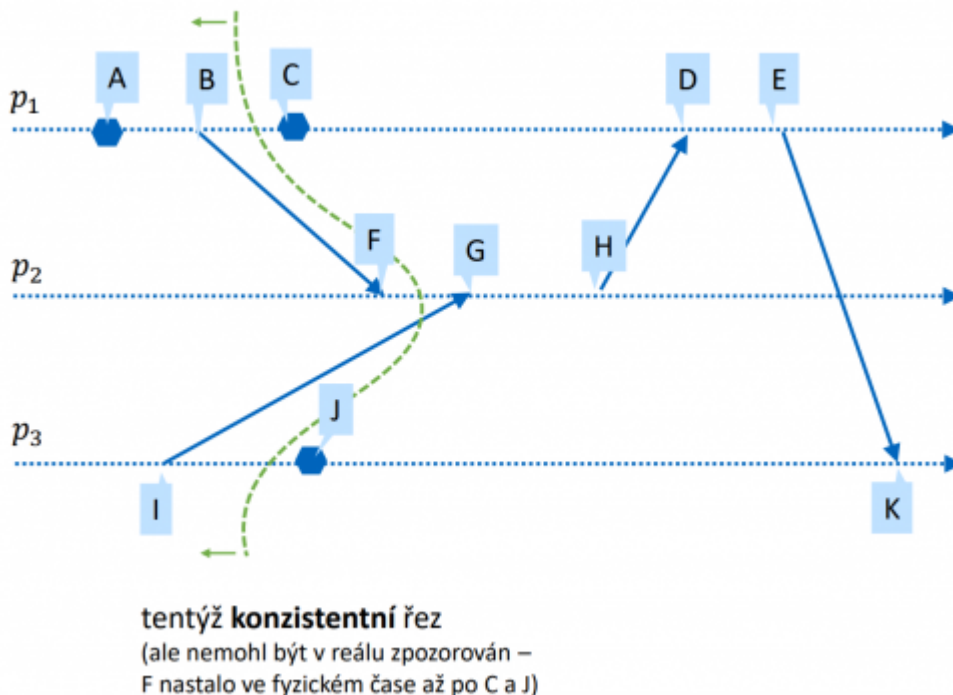
- Nelze, aby byl v řezu příjem zprávy, ale ne její odeslání.

Konzistentní řez je tedy **logický okamžik**, který by mohl být zpozorován „zvenčí“.



Konzistentní globální snapshot odpovídá konzistentnímu řezu.

- Globální snapshot je konzistentní, pokud by mohl (ale nikoliv musel) být zpozorován externím pozorovatelem daného distribuovaného výpočtu.
- Nekonzistentní řez by nemohl nikdy být zpozorován externím pozorovatelem daného distribuovaného výpočtu.



Algoritmus pro distribuovaný snapshot (Chandy-Lamport)

Chandy-Lamport algoritmus umožňuje **distribuovaným procesům získat konzistentní snapshot**, aniž by existovalo globální hodiny.

Základní princip:

- Procesy si **lokálně zaznamenají** svůj stav.
- Stav kanálů mezi nimi se zjistí pomocí **speciální zprávy**: ZNAČKA ■.
- Algoritmus je **asynchronní** a **neblokující** – každý proces koná na základě přijatých značek.

Průběh:

1. Jeden proces iniciuje snapshot – uloží si stav a pošle ZNAČKA ■ všem sousedům.
2. Když jiný proces obdrží ZNAČKA ■ poprvé:
 - uloží svůj stav,
 - pošle ZNAČKA ■ dál,
 - vše, co do té doby přišlo na kanálu, **patří do snapshotu**.
3. Pokud přijde ZNAČKA ■ na kanál, u kterého už máme stav procesu, ale ne kanálu → zprávy mezi tím patří do snapshotu.

Výsledkem je **konzistentní globální snapshot**.

 ilustrace algoritmu slide 15-21

Stabilní vlastnosti

Stabilní vlastnost je taková, že když začne platit, **zůstává platná navždy**.

- Příklady:
 - výpočet skončil
 - došlo k uváznutí
 - objekt je sirotek (už nikdy nebude dosažen)

Nestabilní vlastnost:

- např. „žádný proces neselhal“ – může přestat platit.

Kontrola stabilní vlastnosti pomocí snapshotu

Chandy-Lamport algoritmus lze využít pro **detekci stabilních vlastností**:

Chceme vědět, jestli nějaká stabilní vlastnost $\mathcal{V}$ v systému **nastala**.

- Pokud **platí v globálním snapshotu**, pak:
 - Platila už i ve skutečném systému v okamžiku **ukončení** snapshotu.
- Pokud **neplatí v snapshotu**, pak:
 - Nemohla platit ani v **okamžiku zahájení** snapshotu.

Snapshot tedy vytváří **spolehlivý časový úsek**, během kterého můžeme ověřit trvalost vybraných vlastností.

Vzájemné vyloučení procesů v DS

algoritmy pro vyloučení procesů a jejich vlastnosti

V distribuovaných systémech nemáme sdílenou paměť ani centrální plánovač. Pokud chceme zajistit, že do tzv. **kritické sekce** vstupuje vždy jen jeden proces, potřebujeme speciální algoritmy pro **vzájemné vyloučení**. Tato kapitola popisuje základní problém, požadavky na jeho řešení a několik klíčových algoritmů včetně jejich výhod a nevýhod.

Problém vzájemného vyloučení

- Kritická sekce je část programu, do které smí vstoupit v daný okamžik právě jeden proces – např. pro aktualizaci sdíleného stavu.
- V distribuovaném prostředí je nutné **synchronizovat přístup přes zprávy**.
- Algoritmus poskytuje dvě základní operace:
 - `enter()` – požádá o vstup do kritické sekce
 - `exit()` – opustí kritickou sekci

Požadavky na algoritmus pro vyloučení procesů

- **Bezpečnost**: nikdy není více než jeden proces v kritické sekci současně.
- **Živost**: každý požadavek na vstup do kritické sekce je časem uspokojen.
- **Uspořádání (volitelné)**: pokud jeden požadavek kauzálně předchází jinému, měl by být obsloužen dříve.
- Předpoklady:
 - procesy neselehávají
 - komunikace je spolehlivá (FIFO, bez ztrát, bez duplicit)
 - systém je **asynchronní** s konečnou latencí

Analýza výkonnosti algoritmů

- **Komunikační zátěž** – kolik zpráv je potřeba pro každý vstup/výstup do KS
- **Zpoždění klienta** – čas od požadavku po vstup, pokud nikdo jiný nečeká
- **Synchronizační zpoždění** – čas mezi výstupem jednoho procesu a vstupem dalšího

Centralizovaný algoritmus

- Jeden proces slouží jako **koordinátor** a spravuje **token**.
- Proces žádá koordinátora o token; po jeho získání vstoupí do kritické sekce.
- Po výstupu token vrací.
- Token je předáván podle fronty požadavků.

Vlastnosti:

- Bezpečnost i živost zaručena.
- Komunikační zátěž: 2 zprávy (žádost + token) pro vstup, 1 pro výstup.
- Zpoždění klienta: 2 latence.
- Synchronizační zpoždění: 2 latence.
- Nevýhoda: **koordinátor je single point of failure**.

Kruhový algoritmus

- Procesy jsou uspořádány v logickém kruhu.
- Jeden **token** obíhá dokola.
- Proces vstupuje do kritické sekce až po přijetí tokenu.
- Pokud token nepotřebuje, rovnou ho předá dál.

Vlastnosti:

- Komunikační zátěž: až N zpráv při čekání na token.
- Zpoždění klienta: 0 až N latencí.
- Synchronizační zpoždění: 1 až $N - 1$ latencí.
- Výhoda: žádný centrální uzel.
- Nevýhoda: token může „zabloudit“ nebo se ztratit.

Ricart-Agrawalův algoritmus

- Nepoužívá token – pracuje pomocí **multicastu a logických hodin**.
- Každý proces si udržuje:
 - stav (*RELEASED*, *WANTED*, *HELD*)
 - frontu odložených žádostí
- Pro vstup do kritické sekce:
 - proces pošle **REQUEST** všem, zaznamená čas
 - čeká na odpovědi **OK** od všech
- Ostatní procesy odpoví podle kauzálního (Lamportova) času

```
// schéma odpovědi na REQUEST(K)
if (stav == HELD || (stav == WANTED && (můj_čas < jejich_čas || (můj_čas == jejich_čas && můj_id < jej
odlož požadavek;
} else {
    pošli OK;
}
```

- Po ukončení práce v kritické sekci proces:
 - přepne stav na RELEASED
 - pošle OK na všechny odložené požadavky

Vlastnosti:

- Komunikační zátěž:
 - $2(N - 1)$ zpráv při vstupu (REQUEST + OK)
 - až $(N - 1)$ zpráv při výstupu (zpracování fronty)
- Zpoždění klienta: 2 latence
- Synchronizační zpoždění: 1 latence
- Výhoda: **zcela decentralizovaný**, deterministický
- Nevýhoda: **vysoký počet zpráv**

Volba lídra v DS

algoritmy pro volbu lídra a jejich vlastnosti

V distribuovaném systému často potřebujeme zvolit jeden proces, který bude působit jako koordinátor – například pro synchronizaci, správu zámek nebo organizaci úkolů. Jelikož uzly mohou selhávat, musí volba probíhat robustně a bez centrální autority.

Cílem algoritmu pro **volbu lídra** je zajistit, aby:

- všechny živé procesy se **shodly na jednom lídrovi**,
- nezáleželo na tom, kdo volbu zahájil,
- více paralelních voleb nakonec **konvergovalo k jedné**.

Předpokládáme:

- spolehlivé doručování zpráv (žádné ztráty ani duplicity),
- možnost selhání procesů (typicky crash),
- procesy mají unikátní ID (číselné) a dokážou je mezi sebou porovnat.

Kruhový algoritmus (Ring)

- Procesy jsou logicky uspořádány do kruhu (např. podle ID). Každý zná svého následníka a umí ho kontaktovat.
- Při podezření na pád lídra (např. po timeoutu) proces P_i zahájí volby:
 - pošle **ELECTION**(i) svému následníkovi.
 - pokud příjemce dostane **ELECTION**(j):
 - pokud $j > i$, předá dál beze změny,
 - pokud $j < i$, nahradí vlastním ID a pošle dál,
 - pokud $j = i$, zpráva oběhla kruh $\Rightarrow P_i$ má nejvyšší ID $\Rightarrow$ vítězí.
- Nový lídr pak vyše **ELECTED**(i) kolem kruhu $\rightarrow$ všichni si uloží nového lídra.

Vlastnosti:

- Komunikační složitost: $\mathcal{O}(n)$
- Funguje správně i při více paralelních volbách
- Zvítězí proces s nejvyšším ID
- Nevyžaduje globální znalost všech procesů

Algoritmus Bully

- Procesy jsou propojeny do **úplné sítě** – každý zná každého.
- Pokud P_i zjistí, že koordinátor mlčí (timeout), zahájí volby:
 1. Pošle **ELECTION** všem **procesům s vyšším ID**
 2. Pokud žádný neodpoví $\rightarrow P_i$ se **prohlásí lídrem**
 3. Pošle **COORDINATOR**(i) všem **nižším ID**
 4. Pokud někdo odpoví **OK** $\rightarrow P_i$ čeká na **COORDINATOR**(k) od silnějšího
 5. Pokud neobdrží oznámení včas, znovu zahájí volby

Zprávy:

- **ELECTION** – „ozvi se, pokud žiješ a jsi silnější“
- **OK** – potvrzení, že někdo silnější žije
- **COORDINATOR**(k) – oznámení o novém lídrovi

Vlastnosti:

- Zaručuje, že vítězí **nejvyšší živé ID**
- Horší složitost: $\mathcal{O}(n^2)$ zpráv v nejhorším případě
- Rychlejší než kruh, pokud je vyšší ID blízko
- Vyžaduje spolehlivou detekci pádů (není odolný vůči partitioning)

Trace: • [b4b36pdv](#)